Relatório para Projeto de Sistemas Distribuídos

Autenticação Multifator

Diogo Soares

Nº20201031

Este projeto teve como objetivo desenvolver uma Web APP tolerante a faltas que utilize um sistema de autenticação multifator com recurso ao standard da Google (Google Authenticator) de forma a tornar a mesma mais segura.

Para atingir este objetivo foi necessária a implementação de várias tecnologias e de uma arquitetura bem definida.

Neste projeto foram implementadas as seguintes tecnologias:

- **Docker** – O Docker é utilizado para criar todos os containers da Web APP, incluindo SpringBoot, Nginx, Postgres e HAProxy;

- **Springboot** – Esta é a tecnologia principal uma vez que desempenha vários papéis de uma só vez, entre os quais:
    - Servidor: O springboot abre um servidor em localhost para aceder ao website em que ocorrem os registos e autenticações;
    - API/Aplicação: Gere os registos, acessos e autenticação através de um front-end comunicando com a base de dados em conjunto com a SpringSecurity;

- **SpringSecurity** – Inserida no Springboot a SpringSecurity gere a autenticação e autorização de utilizadores;

- **Nginx** – O Ngingx vai ser o reverse proxy, responsável pela gestão de tráfego (load balancing) dos containers do SpringBoot;

- **HAProxy** - Apesar de também realizar load balancing, o HAProxy foi incorporado no projeto com o objetivo principal de realizar testes de load balancing.

- **Postgres** – O Postgres é responsável pela gestão da base de dados e da sua replicação tendo uma base de dados master e uma outra slave que caso a master falhe mantém os dados armazenados até esse momento servindo de backup;

Diagrama original da arquitetura:

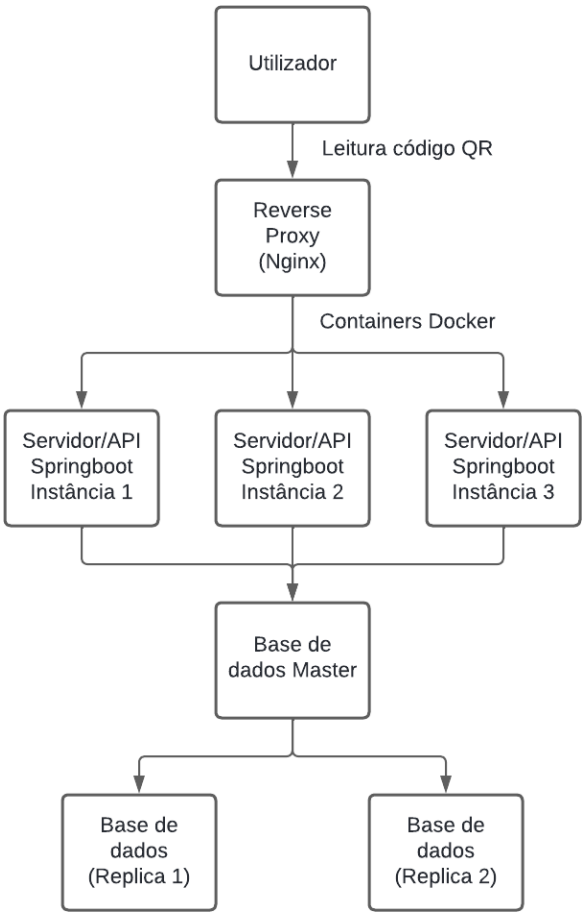
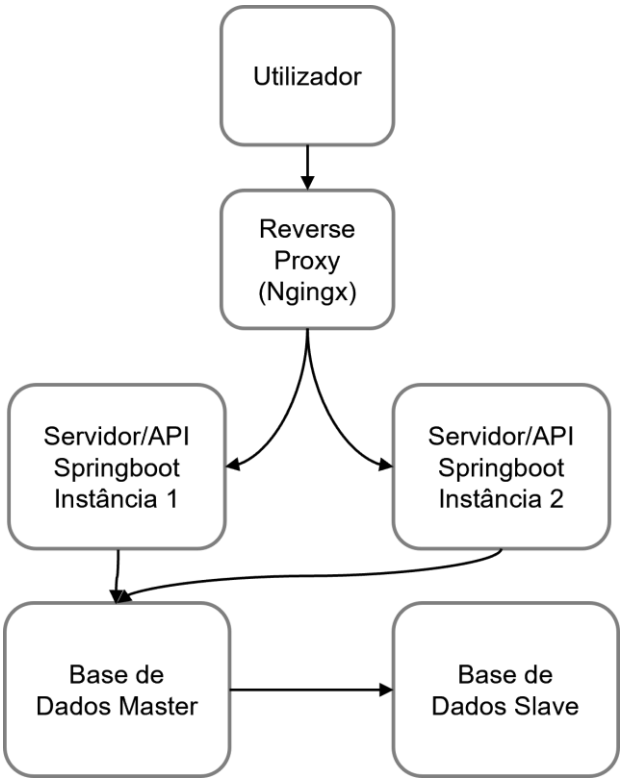


Diagrama final da arquitetura:



Fluxo da aplicação:

1. O utilizador acede ao localhost e o nginx redireciona para o container com menos carga;

2. O utilizador pode realizar o registo e o seu utilizador é registado como NEW_USER tanto na base de dados master como na slave;

3. O código QR e o link alternativo são gerados e o utilizador associa o mesmo ao Google Authenticator;

4. Ao realizar o login o estado do utilizador passa de NEW_USER para PRE_OTP_VERIFIED e é redirecionado para a página de verificação de OTP em que deve inserir o código gerado no Google Authenticator;

5. Ao inserir o código errado a página informa do erro e não permite avançar;

6. Ao inserir o código correto o utilizador ganha acesso à página index e o seu estado passa de PRE_OTP_VERIFIED para OTP_VERIFIED;

7. Caso o utilizador faça logout e queira voltar a fazer login, o seu estado regride de OTP_VERIFIED para PRE_OTP_VERIFIED para voltar a ser redirecionado para a página de verificação de OTP e confirmar a sua identidade;

8. Caso um dos containers do SpringBoot falhe a aplicação continua a funcionar como pretendido uma vez que o nginx redireciona os requests do utilizador para o outro container que está funcional.

Como executar a aplicação:

1. Abrir o docker
2. Abrir a pasta da aplicação num IDE à escolha;
3. Executar no terminal da IDE: “docker build -t authentication-app .”;
4. Executar no terminal da IDE: “docker-compose up --build”;
5. Aceder ao url “localhost” num navegador à escolha.
6. Efetuar o registo e ler o Código QR com o Google Authenticator;
7. Fazer o login e inserir a OTP gerada pelo Google Authenticator;

Link GitHub

https://github.com/Diogocsoaress/TwoFactorAuth